

Universidad del Valle de Guatemala

Visión por Computadora

Sección 10



Proyecto 01

Link de GitHub: <https://github.com/angelcast2002/CV-Proyecto1>

Javier Alejandro Azurdia Arrecis - 21242

Angel Sebastián Castellanos Pineda - 21700

Diego Alejandro Morales Escobár - 21146

Problema 1

Para llevar a cabo la resolución de este problema se utilizaron las siguientes soluciones con sus respectivos resultados:

Otsu + Ecualización

Imagen	Accuracy	Recall	Specificity	Precision	F1 Score
1.pgm	0.477167	0.500661	0.476136	0.040225	0.074467
10.pgm	0.363733	0.003546	0.380258	0.000262	0.000489
11.pgm	0.4679	0.531276	0.465213	0.040421	0.075127
12.pgm	0.322033	0.013737	0.348575	0.001812	0.003202
13.pgm	0.445456	0.236034	0.454131	0.017597	0.032753
14.pgm	0.4512	0.289016	0.463712	0.039917	0.070146
15.pgm	0.353133	0.002264	0.371242	0.000186	0.000343
16.pgm	0.468611	0.335612	0.47639	0.036133	0.065242
17.pgm	0.461811	0.111034	0.479433	0.010601	0.019355
18.pgm	0.457311	0.068661	0.478466	0.007115	0.012894
19.pgm	0.266933	0.001678	0.279824	0.000113	0.000212
2.pgm	0.481167	0.339218	0.488199	0.03179	0.058132
20.pgm	0.431711	0.134356	0.454422	0.018461	0.032462
3.pgm	0.481244	0.376261	0.485829	0.030969	0.057227
4.pgm	0.501278	0.466683	0.502877	0.041596	0.076383
5.pgm	0.478944	0.382488	0.482564	0.026989	0.05042
6.pgm	0.480533	0.438757	0.482114	0.031068	0.058027
7.pgm	0.440744	0.042212	0.461377	0.004041	0.007376
8.pgm	0.449878	0.099548	0.467023	0.009058	0.016605
9.pgm	0.434522	0.168449	0.448798	0.016132	0.029445
Promedio	0.43576545	0.22707455	0.44732915	0.0202243	0.03701535

El algoritmo utiliza una implementación manual del método de umbralización de Otsu, combinado con un preprocesamiento previo de suavizado Gaussiano (kernel 5x5). Para cada imagen en escala de grises, primero se aplica un suavizado para reducir el ruido. Luego, el método Otsu determina automáticamente el umbral global óptimo separando la imagen en píxeles de primer plano y fondo, utilizando varianzas acumulativas de los valores de intensidad. Con ese umbral global, la imagen es binarizada asignando píxeles menores al umbral como blanco (255) y el resto como negro (0). Finalmente, las imágenes umbralizadas se comparan con las imágenes de referencia (ground truth) para calcular métricas de evaluación.

Los resultados indican un desempeño considerablemente bajo del método Otsu en esta implementación, reflejado en valores bajos en todas las métricas, especialmente Recall (promedio ~23%) y Precisión (promedio ~2%), generando muchos falsos positivos y negativos. Aunque Otsu es conocido por ser efectivo en situaciones con distribuciones bimodales claras, aquí su desempeño se ve afectado posiblemente por condiciones de iluminación irregular o por características no bimodales en las imágenes, resultando en una baja detección del objeto objetivo y muy poca fiabilidad en la segmentación.

Sauvola

Imagen	Accuracy	Recall	Specificity	Precision	F1 Score
1.pgm	0.9472	0.185401	0.980608	0.295407	0.227819
10.pgm	0.878622	0.116008	0.91361	0.058033	0.077365
11.pgm	0.941811	0.215242	0.97262	0.25	0.231322
12.pgm	0.923344	0.252733	0.981078	0.534856	0.343265
13.pgm	0.967244	0.650559	0.980363	0.57849	0.612411
14.pgm	0.936789	0.145672	0.997822	0.837645	0.248183
15.pgm	0.958678	0.430383	0.985943	0.612436	0.505518
16.pgm	0.965711	0.532676	0.991038	0.776605	0.631918
17.pgm	0.917856	0.047154	0.961596	0.0581	0.052058
18.pgm	0.919944	0.087602	0.965251	0.120664	0.101509
19.pgm	0.954644	0.021338	1	1	0.041784
2.pgm	0.939522	0.220574	0.975138	0.305311	0.256116
20.pgm	0.9479	0.434858	0.987084	0.71999	0.542224
3.pgm	0.953111	0.231014	0.984646	0.396536	0.291946
4.pgm	0.958522	0.064622	0.999849	0.951852	0.121027
5.pgm	0.949733	0.407066	0.970096	0.338096	0.369389
6.pgm	0.945133	0.355271	0.967458	0.292377	0.32077
7.pgm	0.972556	0.561174	0.993853	0.825365	0.6681
8.pgm	0.941089	0.097404	0.982378	0.21291	0.13366
9.pgm	0.947167	0.592407	0.966201	0.484648	0.533137
Promedio	0.9433288	0.2824579	0.9778316	0.48246605	0.31547605

El algoritmo implementado aplica una umbralización adaptativa utilizando el método manual de Sauvola. Las imágenes se cargan en escala de grises y luego se suavizan con un filtro Gaussiano de tamaño 5x5 para reducir ruido. Después, para cada píxel, se calcula un umbral dinámico basado en la media y desviación estándar local del bloque (31x31) que rodea al píxel, ajustado por un factor $k=0.2$ y una constante $R=128$. Finalmente, se convierte cada píxel a blanco o negro dependiendo de si su intensidad está por debajo del umbral calculado. Posteriormente, se evalúan las imágenes resultantes contra etiquetas de referencia para obtener métricas de evaluación.

La implementación de Sauvola presenta dificultades importantes para detectar correctamente las regiones objetivo (bajo recall). Aunque globalmente el accuracy es alto, no refleja el desempeño real del algoritmo por el fuerte desequilibrio en falsos negativos.

Umbralización Adaptativa

Imagen	Accuracy	Recall	Specificity	Precision	F1 Score
1.pgm	0.907844	0.91087	0.907712	0.302079	0.453695
10.pgm	0.913033	0.847011	0.916062	0.316457	0.460765
11.pgm	0.861611	0.840481	0.862507	0.205847	0.3307
12.pgm	0.917656	0.924586	0.917059	0.489717	0.640295

13.pgm	0.8901	0.906983	0.889401	0.253573	0.396338
14.pgm	0.920722	0.873565	0.92436	0.471174	0.612165
15.pgm	0.936489	0.935477	0.936541	0.432082	0.59113
16.pgm	0.859944	0.932837	0.855681	0.274335	0.423982
17.pgm	0.900011	0.854355	0.902305	0.305228	0.449771
18.pgm	0.893289	0.817262	0.897427	0.302502	0.441563
19.pgm	0.962167	0.861664	0.967051	0.559639	0.678561
2.pgm	0.882733	0.917608	0.881006	0.276415	0.42485
20.pgm	0.894389	0.95036	0.890114	0.397785	0.560828
3.pgm	0.896522	0.93282	0.894937	0.279408	0.430014
4.pgm	0.910689	0.733216	0.918894	0.294754	0.420476
5.pgm	0.876344	0.868203	0.87665	0.208931	0.336809
6.pgm	0.865867	0.87081	0.86568	0.197022	0.32134
7.pgm	0.900533	0.883521	0.901414	0.316923	0.466508
8.pgm	0.905311	0.883544	0.906376	0.315933	0.465437
9.pgm	0.8689	0.879555	0.868328	0.263843	0.405921
Promedio	0.8982077	0.8812364	0.89897525	0.32318235	0.4655574

El algoritmo primero suaviza la imagen utilizando un filtro Gaussiano de kernel 5x5. Luego aplica una umbralización adaptativa manual, donde cada píxel se compara contra la media local calculado en un bloque alrededor de él (tamaño especificado por blockSize). Si el valor del píxel supera esta media local (ajustada con un valor C), el píxel se asigna a blanco o negro según el tipo de umbral (THRESH_BINARY o THRESH_BINARY_INV). Finalmente, se comparan los resultados contra imágenes de referencia para obtener métricas de desempeño.

Los resultados muestran que, aunque la precisión general (Accuracy) es alta (cercana al 90%), el algoritmo presenta problemas con la precisión específica, es decir, genera demasiados falsos positivos. Esto se refleja en un valor bajo del F1-score, que oscila alrededor de 0.42 en promedio. Aunque el algoritmo es efectivo para identificar correctamente gran parte de los píxeles relevantes (alto Recall), la baja precisión indica que está identificando como positivos muchos píxeles que realmente no pertenecen a la región objetivo.

Filtros de Gabor con Red Neuronal (150 épocas)

Métrica	Accuracy	Recall	Specificity	Precision	F1 Score
Valor Promedio	0.9653	0.3398	0.9978	0.8903	0.4723

El algoritmo combina una capa inicial de filtros de Gabor generados manualmente con una red neuronal convolucional para segmentar imágenes. Primero, cada imagen se convierte a escala de grises y es suavizada con un filtro Gaussiano (kernel 5x5). Luego se aplica una capa convolucional con filtros Gabor predefinidos (31x31) en varias orientaciones, cuyos pesos pueden ajustarse durante el entrenamiento. La salida de esta capa pasa por funciones de activación ReLU, normalización por lotes (Batch Normalization), más capas convolucionales

adicionales, pooling y upsampling. La red genera finalmente una máscara binaria (mediante activación sigmoide) que indica qué píxeles pertenecen a la región objetivo. El modelo se entrenó durante 150 épocas usando la función de pérdida `binary_crossentropy` y el optimizador `adam`.

Los resultados obtenidos reflejan un alto valor promedio de Accuracy (96.5%) y una excelente Specificity (99.7%), lo que indica que el modelo es efectivo detectando correctamente los píxeles de fondo. Sin embargo, el Recall es bajo (33.98%), lo que muestra que aún no detecta adecuadamente todas las regiones objetivo, generando falsos negativos. Por otro lado, la Precisión (89.03%) es alta, sugiriendo que cuando la red predice un píxel como parte del objeto, generalmente acierta. El valor del F1-score (47.23%), influenciado por el bajo Recall, señala la necesidad de mejorar la capacidad del modelo para identificar todas las regiones objetivo, posiblemente ajustando hiperparámetros, aumentando la complejidad del modelo o empleando técnicas como data augmentation.

Resultados Finales

Algoritmo	Accuracy	Recall	Specificity	Precision	F1 Score
Otsu + Ecuación	0.43	0.22	0.44	0.02	0.03
Sauvola	0.94	0.28	0.97	0.48	0.31
Umbralización Adaptativa	0.89	0.88	0.89	0.32	0.46
Filtros de Gabor + Red Neuronal	0.96	0.33	0.99	0.89	0.47

Tras analizar los distintos métodos de binarización, se concluye que la combinación de Filtros de Gabor con una red neuronal ofrece los mejores resultados en términos de Precisión (0.89), Accuracy (0.96) y F1 Score (0.47). Aunque algunos métodos como la umbralización adaptativa presentan un recall más alto (0.88), este enfoque logra un balance óptimo entre detectar la estructura arterial y minimizar falsos positivos. La capacidad de los filtros de Gabor para extraer información de textura y la optimización de sus parámetros mediante el entrenamiento de una red neuronal permiten obtener una segmentación más precisa y robusta en diferentes condiciones de iluminación y contraste.

Sin embargo, esta metodología es la más compleja y costosa computacionalmente debido a la necesidad de entrenar una red neuronal. Para este trabajo, se realizaron 150 épocas de entrenamiento en CPU, lo que tomó aproximadamente 15 minutos en procesar las 20 imágenes. Si bien este tiempo es manejable, el entrenamiento puede volverse considerablemente más lento en conjuntos de datos más grandes o con redes más profundas. Además, si no se dispone de una GPU NVIDIA compatible con CUDA, el entrenamiento se ejecutará exclusivamente en la CPU, aumentando significativamente los tiempos de cómputo y limitando su eficiencia en entornos sin hardware especializado.

En conclusión, aunque el método basado en filtros de Gabor y redes neuronales ofrece la segmentación más efectiva, su implementación requiere un mayor nivel de programación y recursos computacionales en comparación con los métodos tradicionales. Para proyectos con restricciones de hardware o tiempos de procesamiento más ajustados, métodos como la

umbralización adaptativa (Accuracy 0.89, F1 0.46) podrían ser una alternativa viable, aunque con menor precisión. Sin embargo, si se dispone de los recursos adecuados, la técnica basada en filtros de Gabor con red neuronal es la opción más recomendada por la calidad de los resultados obtenidos.

Problema 2

El algoritmo desarrollado aborda el problema de convertir imágenes en escala de grises en una representación gráfica mediante grafos. Para ello, se parte de la esqueletonización de la imagen, lo que permite reducir las líneas a un solo píxel de grosor, facilitando la detección precisa de la estructura subyacente.

Inicialmente, cada imagen se carga en escala de grises y se somete a un proceso de umbralización para obtener una imagen binaria. Posteriormente, se aplica la técnica de esqueletonización para reducir el trazado a una forma mínima, eliminando redundancias y dejando solo la “espina dorsal” de la estructura. Este preprocesamiento es de gran ayuda para simplificar el análisis y garantizar que la representación del grafo refleje con precisión la geometría de la imagen.

Una vez obtenido el esqueleto, el algoritmo construye un grafo en el cual cada píxel perteneciente a la línea es considerado un nodo. Mediante la conectividad de 8 vecinos se establecen las aristas que unen dichos nodos, modelando así la continuidad de la estructura.

Para identificar los puntos de mayor relevancia, se seleccionan los nodos “candidatos”, los cuales son aquellos con un grado distinto de 2, ya que representan tanto puntos extremos (grado 1) como intersecciones o ramificaciones (grado mayor o igual a 3).

Con los nodos candidatos identificados, se realiza una búsqueda en profundidad (DFS) para extraer los caminos que conectan dichos nodos, siguiendo los puntos intermedios (nodos de grado 2) hasta alcanzar otro nodo candidato.

Una vez definidos estos caminos, se aplica el algoritmo Ramer-Douglas-Peucker (RDP) para simplificar las trayectorias. Esta técnica elimina puntos redundantes, manteniendo únicamente aquellos que definen la forma general de la trayectoria, lo que resulta crucial para optimizar la estructura del grafo sin perder información esencial.

Dado que pequeñas variaciones o ruido pueden generar múltiples nodos cercanos que representan el mismo punto clave, se emplea el algoritmo DBSCAN para agrupar estos nodos.

Cada cluster se resume mediante su centroide, lo que permite “fusionar” nodos muy próximos y obtener una representación más robusta.

Además, se clasifica cada nodo final según su grado, diferenciando entre nodos extremos, bifurcaciones y trifurcaciones, lo que facilita la interpretación y análisis posterior del grafo.

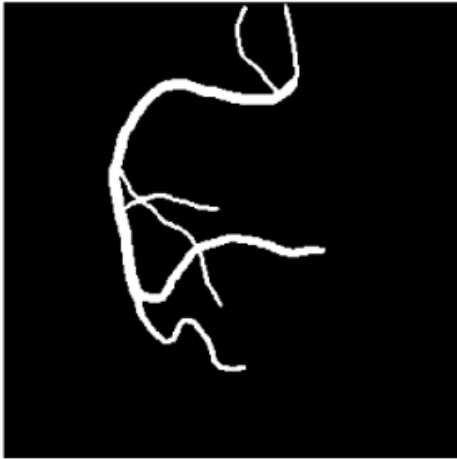
Finalmente, se genera una imagen de visualización en la que se superpone el grafo a la imagen original. Las aristas se dibujan como líneas y los nodos se representan con círculos de colores diferenciados según su clasificación. La estructura del grafo se exporta en formato JSON, mientras que la imagen resultante se guarda en formato PNG.

En conclusión, el enfoque propuesto combina técnicas clásicas de procesamiento de imágenes, como la esqueletonización y la búsqueda en grafos, con métodos de optimización y simplificación (RDP y DBSCAN) para extraer de manera eficiente la estructura topológica de las imágenes. Esta metodología facilita la conversión de trazos complejos en grafos estructurados, ofreciendo una herramienta robusta para aplicaciones en visión por computadora que requieran un análisis detallado de las conexiones y patrones presentes en las imágenes.

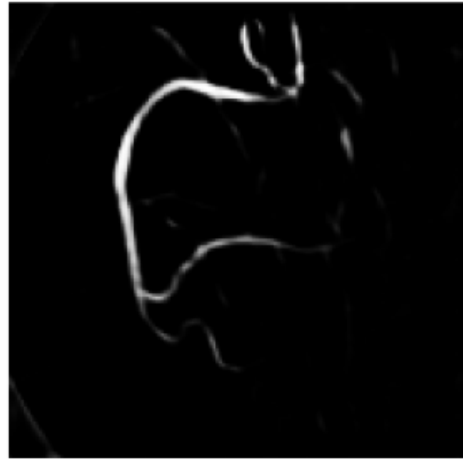
Resultados obtenidos

Parte 1

Ground Truth



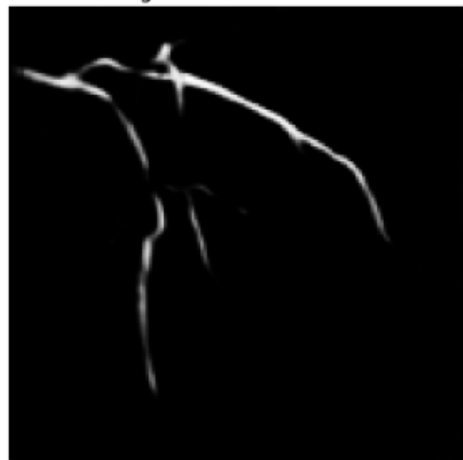
Segmentación Predicha



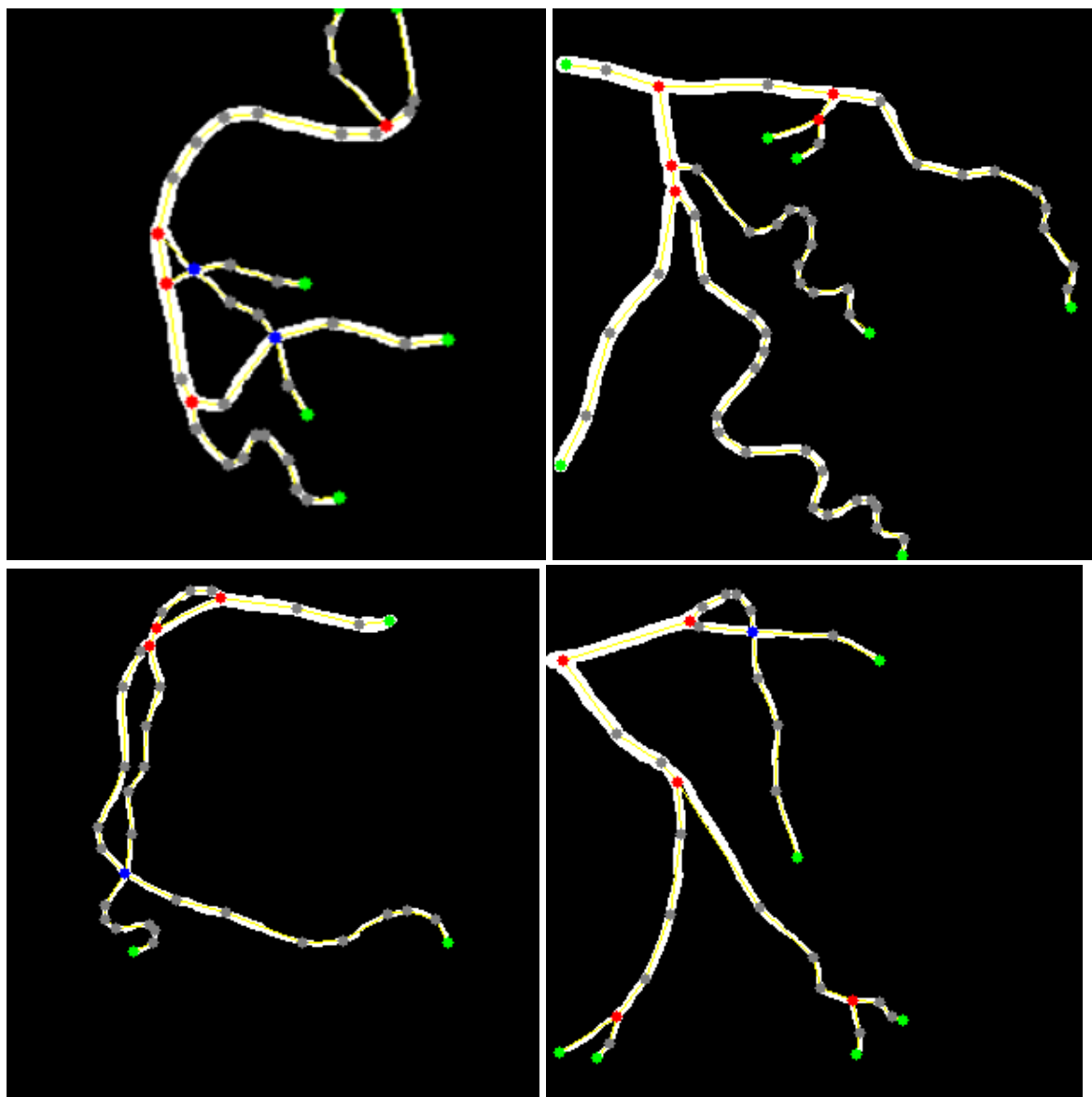
Ground Truth



Segmentación Predicha



Parte 2



Referencias

M, L. V. B., J, M. a. R., & E, I. M. M. (n.d.). *Implementación del Algoritmo de Otsu sobre FPGA*.

http://scielo.sld.cu/scielo.php?script=sci_arttext&pid=S2227-18992016000300002&lng=es&nrm=iso&tlng=es

Principio e implementación del algoritmo OTSU (método Otsu: método de varianza máxima entre clases) - programador clic. (n.d.).

<https://programmerclick.com/article/47001069656/>

Implementación en Python del filtro Gabor - programador clic. (n.d.).

<https://programmerclick.com/article/79311005604/>

Niño-Rondón, C. V., Castro-Casadiegos, S. A., Medina-Delgado, B., Guevara-Ibarra, D., & Camargo-Ariza, L. L. (2021). Comparativa entre la técnica de umbralización binaria y el método de Otsu para la detección de personas. *Revista UIS Ingenierías*, 20(2).

<https://doi.org/10.18273/revuin.v20n2-2021006>

colaboradores de Wikipedia. (2022, 12 noviembre). *Algoritmo de Ramer–Douglas–Peucker*.

Wikipedia, la Enciclopedia Libre.

https://es.wikipedia.org/wiki/Algoritmo_de_Ramer%E2%80%93Douglas%E2%80%93Peucker

colaboradores de Wikipedia. (2024, 24 julio). *DBSCAN*. Wikipedia, la Enciclopedia Libre.

<https://es.wikipedia.org/wiki/DBSCAN>

Esqueletización de una imagen. (2024). En *SCRIBID*. Recuperado 13 de marzo de 2025, de

<https://es.scribd.com/document/444850937/Esqueletizacion-de-una-imagen>